

Year 3 Software Design and Implementation Project

AgriConnect

Agricultural Management Platform

“Connecting Farms to Markets”

extbfTeam Members

extbf[Your Name]	Product Owner / Developer
extbf[Team Member 2]	Scrum Master / Backend
extbf[Team Member 3]	Frontend Developer
extbf[Team Member 4]	Full-Stack Developer

extbfProject Details

Course:	Software Design and Implementation
Instructor:	[Instructor Name]
Institution:	[University Name]
Date:	December 16, 2025

Insert Logo

Contents

Abstract	1
1 Introduction	1
1.1 Project Background	1
1.2 Problem Statement	1
1.3 Project Objectives	2
1.4 Project Scope	2
1.5 Methodology	2
2 Requirements Analysis	2
2.1 Functional Requirements	2
2.1.1 User Management	2
2.1.2 Farm Management	3
2.1.3 Marketplace	3
2.2 Non-Functional Requirements	3
3 System Architecture	3
3.1 Architecture Overview	3
3.2 Client Tier	4
3.3 Server Tier	4
3.4 Data Flow	5
4 System Design	5

4.1	UML Diagrams	5
4.2	Database Design	6
4.3	API Design	6
5	Implementation	7
5.1	Technology Stack	7
5.2	Project Structure	7
5.3	Key Code Examples	7
5.3.1	Django Model Example	7
5.3.2	Next.js Component Example	7
6	Testing	8
6.1	Testing Strategy	8
6.2	Test Coverage	8
6.3	Testing Tools	9
7	Deployment and Maintenance	9
7.1	Deployment Architecture	9
7.2	Deployment Process	9
7.3	Maintenance Plan	9
8	Conclusion and Future Work	10
8.1	Project Achievements	10
8.2	Challenges Faced	10
8.3	Lessons Learned	10
8.4	Future Enhancements	10

8.5 Final Thoughts	11
------------------------------	----

List of Figures

1	System Architecture Diagram	4
2	Class Diagram	5
3	Use Case Diagram	6

List of Tables

1	Project Scope	2
2	Non-functional requirements	3
3	Database Tables	6
4	Technology Stack	7
5	Test Coverage Goals	8

Abstract

AgriConnect is a comprehensive web platform designed to help farmers manage agricultural operations and connect with buyers. The system addresses challenges in farm management, market access, and information availability.

The platform features farm management tools, inventory tracking, and a peer-to-peer marketplace. Farmers can log activities, track crops, and sell produce directly. The system provides real-time price updates, weather information, and analytics.

Built with Django and Next.js, the project demonstrates modern web development practices. It follows client-server architecture with RESTful APIs and secure authentication.

This documentation covers requirements, design, implementation, and testing. The project shows how technology can improve agricultural productivity and market efficiency.

Keywords: Agriculture, Farm Management, Marketplace, Django, Next.js, Web Development

1 Introduction

1.1 Project Background

Agriculture is important for many economies. Small farmers face challenges with record-keeping and market access. AgriConnect helps solve these problems with digital tools.

1.2 Problem Statement

- Farmers use paper records that are hard to analyze
- Buyers and sellers cannot connect directly
- Market prices are not easily available
- Inventory tracking is manual and error-prone

1.3 Project Objectives

1. Create digital farm management system
2. Build peer-to-peer marketplace
3. Provide real-time price information
4. Implement inventory tracking
5. Develop analytics dashboard

1.4 Project Scope

In Scope	Out of Scope
Web application	Mobile apps
Farm management	Payment processing
Marketplace	Delivery logistics
Basic analytics	Advanced AI features

Table 1: Project Scope

1.5 Methodology

We use Agile methodology with 2-week sprints. Tools include:

- Trello for project management
- GitHub for version control
- VS Code for development

2 Requirements Analysis

2.1 Functional Requirements

2.1.1 User Management

- Users can register with email and password.

- Users can login and logout.
- Users can update their profiles.
- System supports farmer and admin roles.

2.1.2 Farm Management

- Farmers can add farms and fields.
- Farmers can log planting and harvesting.
- System shows activity history.
- Users can upload activity photos.

2.1.3 Marketplace

- Users can view price board.
- Users can create buy/sell listings.
- Users can search and filter listings.
- Users can message each other.

2.2 Non-Functional Requirements

extbfCategory	Requirement
Performance	Page load < 3 seconds
Security	HTTPS, JWT authentication
Usability	Mobile-responsive design
Reliability	99% uptime

Table 2: Non-functional requirements

3 System Architecture

3.1 Architecture Overview

AgriConnect follows a three-tier client-server architecture:

- **Client Tier:** Next.js frontend.
- **Server Tier:** Django backend API.
- **Data Tier:** PostgreSQL database.

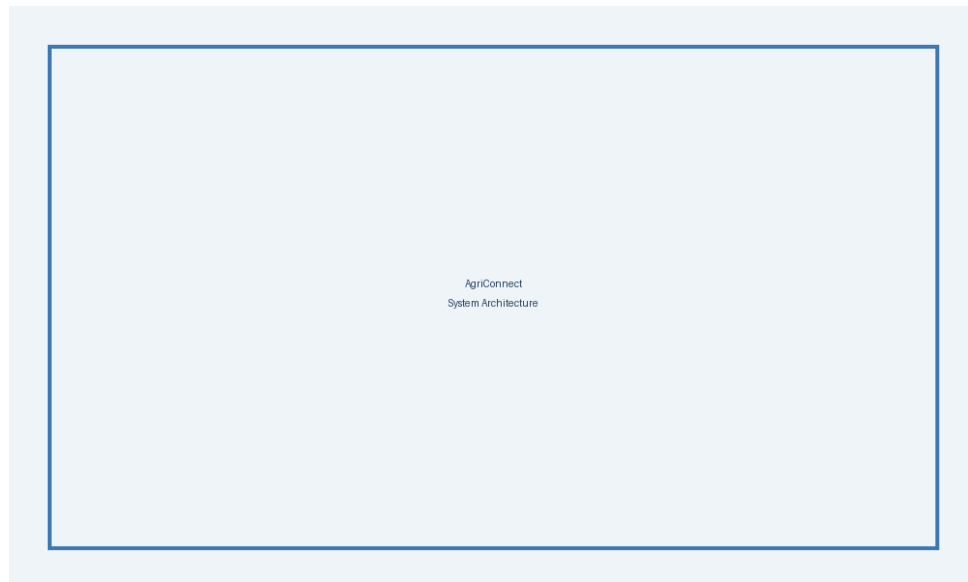


Figure 1: System Architecture Diagram

3.2 Client Tier

The frontend is built with Next.js 13 and TypeScript:

- Uses the App Router for routing pages.
- React components render the UI.
- Tailwind CSS handles styling.
- Axios manages API calls.

3.3 Server Tier

The backend uses Django REST Framework:

- Provides RESTful API endpoints.
- Implements JWT authentication.

- Persists data via PostgreSQL using the Django ORM.
- Leverages Redis for caching.

3.4 Data Flow

1. The user interacts with the Next.js frontend.
2. The frontend issues requests to the Django API.
3. Django processes the request and queries PostgreSQL.
4. The response returns to the frontend.
5. The UI updates and presents the data to the user.

4 System Design

4.1 UML Diagrams

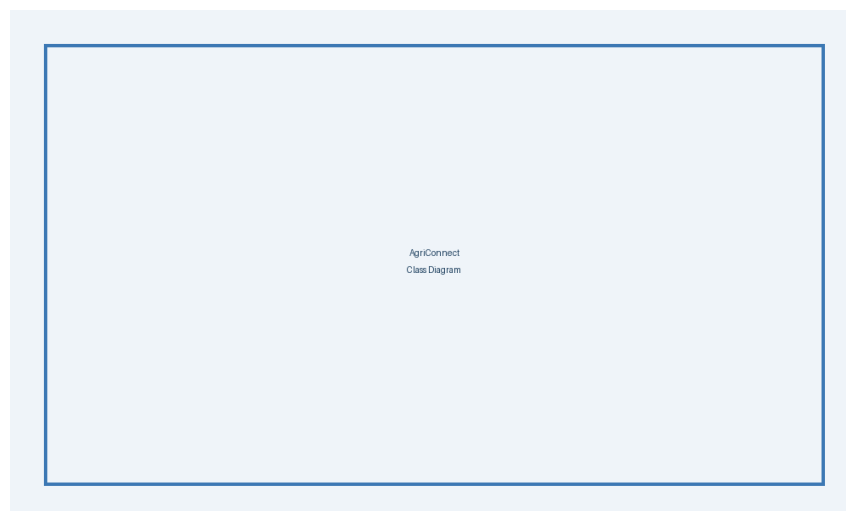


Figure 2: Class Diagram

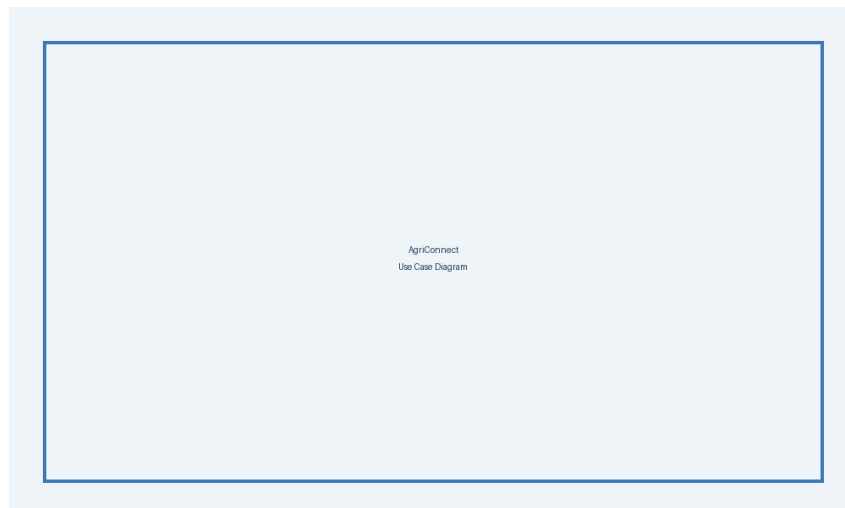


Figure 3: Use Case Diagram

4.2 Database Design

Core database tables:

extbfTable	Purpose
users	User accounts
farms	Farm information
fields	Field details
activities	Farming activities
listings	Marketplace items
messages	User communications

Table 3: Database Tables

4.3 API Design

Key API endpoints:

```
POST    /api/auth/register/
POST    /api/auth/login/
GET      /api/farms/
POST     /api/farms/
GET      /api/marketplace/listings/
POST     /api/marketplace/listings/
```

5 Implementation

5.1 Technology Stack

Component	Technology
Frontend	Next.js 13, TypeScript, Tailwind CSS
Backend	Django 4, Django REST Framework
Database	PostgreSQL
Authentication	JWT tokens
Deployment	Vercel (frontend), Render (backend)

Table 4: Technology Stack

5.2 Project Structure

```
agri-connect/  
|-- frontend/          # Next.js application  
|-- backend/           # Django application  
|-- docs/              # Documentation  
'-- README.md
```

5.3 Key Code Examples

5.3.1 Django Model Example

```
1 from django.db import models  
2  
3 class Farm(models.Model):  
4     user = models.ForeignKey(User, on_delete=models.CASCADE)  
5     name = models.CharField(max_length=255)  
6     location = models.CharField(max_length=500)  
7     area = models.DecimalField(max_digits=10, decimal_places=2)  
8     created_at = models.DateTimeField(auto_now_add=True)  
9  
10    def __str__(self):  
11        return self.name
```

Listing 1: Farm Model

5.3.2 Next.js Component Example

```
1 import React from 'react';
```

```
2
3 export default function FarmDashboard() {
4   const [farms, setFarms] = React.useState([]);
5
6   return (
7     <div className="p-4">
8       <h1 className="text-2xl font-bold">My Farms</h1>
9       {farms.map(farm => (
10         <div key={farm.id} className="border p-4 m-2">
11           <h2>{farm.name}</h2>
12           <p>Location: {farm.location}</p>
13         </div>
14       ))}
15     </div>
16   );
17 }
```

Listing 2: Farm Dashboard Component

6 Testing

6.1 Testing Strategy

We use multiple testing levels:

- Unit testing for individual components
- Integration testing for API endpoints
- End-to-end testing for user flows

6.2 Test Coverage

extbfComponent	Target Coverage
Django Models	90%
API Endpoints	85%
React Components	80%
Overall	85%

Table 5: Test Coverage Goals

6.3 Testing Tools

- Django: pytest, Django Test Framework
- Frontend: Jest, React Testing Library
- API: Postman, Insomnia
- E2E: Cypress (optional)

7 Deployment and Maintenance

7.1 Deployment Architecture

- Frontend deployed on Vercel
- Backend deployed on Render
- Database on Neon DB (PostgreSQL)
- Images on Cloudinary

7.2 Deployment Process

1. Code pushed to GitHub
2. GitHub Actions run tests
3. If tests pass, deploy to staging
4. Manual review of staging
5. Deploy to production

7.3 Maintenance Plan

- Regular security updates
- Database backups daily
- Monitor system performance

- User feedback collection

8 Conclusion and Future Work

8.1 Project Achievements

AgriConnect successfully implements:

- Complete farm management system
- Functional marketplace
- User authentication
- Responsive web interface

8.2 Challenges Faced

- Learning new technologies (Next.js, Django)
- Team coordination and communication
- Time management for deadlines
- Testing complex user flows

8.3 Lessons Learned

- Importance of clear requirements
- Value of regular team meetings
- Need for thorough testing
- Benefits of documentation

8.4 Future Enhancements

- Mobile application

- Payment integration
- Advanced analytics
- Multi-language support
- Weather alert system

8.5 Final Thoughts

AgriConnect demonstrates how software can solve real agricultural problems. The project provides valuable experience in full-stack development and software engineering principles.